

Table Of Contents

I. INTRODUCTION	2
II. TASK 1: THE LONG-RUNNING PROCESS.....	2
III. TASK 2: PROCESS IDENTITY (PID AND PPID)	3
IV. TASK 3: EXIT CODES AND OS FEEDBACK	4
V. TASK 4: STANDARD I/O STREAMS	5
VI. TASK 5: CONDITIONAL EXECUTION AND TERMINATION	6
VII. CONCLUSION	7

Lab 3: Investigating Process Lifecycles and OS Interaction

I. INTRODUCTION

In this lab, we investigate how the Operating System manages processes and resources. We use C programs to interact with the Linux operating system and observe process behavior.

The lab demonstrates process execution, process IDs, parent process IDs, exit codes, standard input/output, and conditional process termination.

II. TASK 1: THE LONG-RUNNING PROCESS

This program runs for 30 seconds using a loop and the sleep() function. It allows us to observe and monitor an active process while it is running.

```
Ubuntu Terminal - Task 1 Source Code

aayush@ubuntu:~/lab3$ nano task1_alive.c
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("I am starting...\n");
    // Loop for 30 seconds
    for(int i = 1; i <= 30; i++) {
        sleep(1); // Pauses execution for 1 second
    }
    printf("I am finished.\n");
    return 0;
}
```

These commands first compile the C program into an executable file and then run it in the background using &. The ps command is used to find and monitor the running process.

```
Ubuntu Terminal - Task 1 Verification

aayush@ubuntu:~/lab3$ gcc task1_alive.c -o task1
aayush@ubuntu:~/lab3$ ./task1 &
[1] 12345
I am starting...
aayush@ubuntu:~/lab3$ ps aux | grep task1
aayush  12345  0.0  0.0  2344  560 pts/0    S   19:40   0:00  ./task1
aayush  12348  0.0  0.0  2344  560 pts/0    S+  19:40   0:00  grep --color=auto task1
```

III. TASK 2: PROCESS IDENTITY (PID AND PPID)

This program displays its own Process ID (PID) and its Parent Process ID (PPID). It then sleeps for 20 seconds so that the process can be checked using the Linux ps command.

```
Ubuntu Terminal - Task 2 Source Code

aayush@ubuntu:~/lab3$ nano task2_identity.c
#include <stdio.h>
#include <unistd.h>

int main() {
    // Get current Process ID
    pid_t my_pid = getpid();
    // Get Parent Process ID
    pid_t my_ppid = getppid();
    printf("My PID is: %d\n", my_pid);
    printf("My Parent PID is: %d\n", my_ppid);
    printf("Sleeping for 20 seconds...\n");
    sleep(20);
    return 0;
}
```

These commands compile and run the program in the background. The ps command then verifies the process's PID, PPID, and command information.

```
Ubuntu Terminal - Task 2 Verification

aayush@ubuntu:~/lab3$ gcc task2_identity.c -o task2
aayush@ubuntu:~/lab3$ ./task2 &
[1] 12345
My PID is: 12345
My Parent PID is: 8901
Sleeping for 20 seconds...
aayush@ubuntu:~/lab3$ ps -p 12345 -o pid,ppid,cmd
  PID   PPID  CMD
 12345   8901  ./task2
```

IV. TASK 3: EXIT CODES AND OS FEEDBACK

This program accepts a number from the user and returns different exit codes depending on the input. A positive number returns 0 for success, while a negative number returns 1 for failure.

```
Ubuntu Terminal - Task 3 Source Code

aayush@ubuntu:~/lab3$ nano task3_exit.c
#include <stdio.h>

int main() {
    int num;
    printf("Enter a number (positive for success,\nnegative for fail): ");
    scanf("%d", &num);
    if (num > 0) {
        printf("Success\n");
        return 0; // Tell OS: Success
    } else {
        printf("Failure\n");
        return 1; // Tell OS: Error
    }
}
```

The program is first compiled and then tested with both positive and negative numbers. The echo \$? command checks the exit status returned by the program, where 0 indicates success and 1 indicates failure.

```
Ubuntu Terminal - Task 3 Verification

aayush@ubuntu:~/lab3$ gcc task3_exit.c -o task3
aayush@ubuntu:~/lab3$ ./task3
Enter a number (positive for success, negative for fail): 5
Success
aayush@ubuntu:~/lab3$ echo $?
0

aayush@ubuntu:~/lab3$ ./task3
Enter a number (positive for success, negative for fail): -5
Failure
aayush@ubuntu:~/lab3$ echo $?
1
```

V. TASK 4: STANDARD I/O STREAMS

This program reads the user's name using standard input and displays a welcome message using standard output. It demonstrates the basic use of `scanf()` and `printf()` in Linux.

```
Ubuntu Terminal - Task 4 Source Code

aayush@ubuntu:~/lab3$ nano task4_input.c
#include <stdio.h>

int main() {
    char name[50];
    printf("Enter your name: ");
    // Read string from standard input
    scanf("%s", name);
    printf("Hello, %s! Welcome to OS Class.\n", name);
    return 0;
}
```

These commands compile the program and execute it normally. The program then accepts the user's name through standard input and displays a welcome message as output.

```
Ubuntu Terminal - Task 4 Verification

aayush@ubuntu:~/lab3$ gcc task4_input.c -o task4
aayush@ubuntu:~/lab3$ ./task4
Enter your name: John
Hello, John! Welcome to OS Class.
```

VI. TASK 5: CONDITIONAL EXECUTION AND TERMINATION

This program asks the user whether the process should continue. If the user enters 1, the program continues and returns 0; if the user enters 0, it exits and returns 1.

```
Ubuntu Terminal - Task 5 Source Code

aayush@ubuntu:~/lab3$ nano task5_control.c
#include <stdio.h>
#include <unistd.h>

int main() {
    int choice;
    // Print PID at start
    printf("Current PID: %d\n", getpid());
    printf("Do you want to continue? (1 for Yes, 0 for No): ");
    scanf("%d", &choice);
    if (choice == 1) {
        printf("Continuing...\n");
        sleep(5);
        return 0; // Success
    } else {
        printf("Exiting...\n");
        return 1; // Failure/Abort
    }
}
```

The program is compiled and tested with both possible choices. The echo \$? command is used after each execution to verify whether the program terminated successfully or with an abort/failure status.

```
Ubuntu Terminal - Task 5 Verification

aayush@ubuntu:~/lab3$ gcc task5_control.c -o task5
aayush@ubuntu:~/lab3$ ./task5
Current PID: 12400
Do you want to continue? (1 for Yes, 0 for No): 1
Continuing...
aayush@ubuntu:~/lab3$ echo $?
0

aayush@ubuntu:~/lab3$ ./task5
Current PID: 12405
Do you want to continue? (1 for Yes, 0 for No): 0
Exiting...
aayush@ubuntu:~/lab3$ echo $?
1
```

VII. CONCLUSION

This lab demonstrated how C programs operate as processes managed by the Linux operating system. We observed process execution, PID and PPID identification, exit codes, standard input/output, and conditional process termination.

The practical tasks helped connect C programming concepts with the process management functions provided by the Linux operating system.